OS Lab

Practical-11

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
   for (int i = 0; i < ITERATIONS; i++)
   {
      counter++;
   }

   return NULL;
}

int main()
{
   pthread_t threads[NUM_THREADS];

   printf("Initial counter: %d\n", counter);

   // Create threads
   for (int i = 0; i < NUM_THREADS; i++)
   {
      pthread_create(&threads[i], NULL,
                  increment_counter, NULL);
   }
```

```
    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter: %d\n", counter);

    return 0;
}
```

```
Content modified using mmap()!lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ cd ~/OSLab/file_io
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano pthread_race.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc pthread_race.c -o pthread_race -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 175412
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 147510
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 163427
```

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define ITERATIONS 100000

int counter = 0;

pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
```

```
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Initialize mutex
    pthread_mutex_init(&lock, NULL);

    printf("Initial counter: %d\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                       increment_counter, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter: %d\n", counter);

    // Destroy mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano pthread_mutex.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc pthread_mutex.c -o pthread_mutex -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$
```